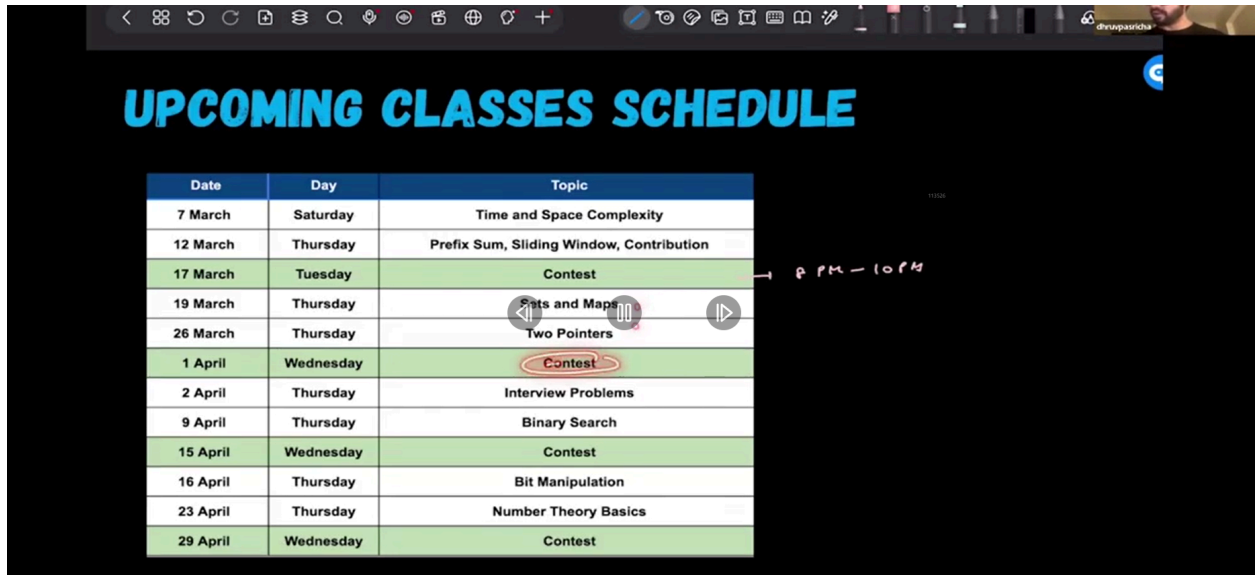


1. Planning for upcoming session

Next contest on Tuesday because of super 30 test which include Times and space complexity and prefix sum, sliding window contribution



Date	Day	Topic
7 March	Saturday	Time and Space Complexity
12 March	Thursday	Prefix Sum, Sliding Window, Contribution
17 March	Tuesday	Contest
19 March	Thursday	Sets and Maps
26 March	Thursday	Two Pointers
1 April	Wednesday	Contest
2 April	Thursday	Interview Problems
9 April	Thursday	Binary Search
15 April	Wednesday	Contest
16 April	Thursday	Bit Manipulation
23 April	Thursday	Number Theory Basics
29 April	Wednesday	Contest

2.

TIME COMPLEXITY

1. Find the # of iterations in terms of input size (N)
2. Remove the constants
3. Remove the lower contribution terms

1) $5N^2 + 3N + 10 \rightarrow O(N^2)$

2) $11N^3 + 12N^2 + 13N + 17000 \rightarrow O(N^3)$

1) $\text{for}(i=1; i \leq N; i++)$
 $\{$
 $\quad \equiv$
 $\}$

$O(N)$

2) $\text{for}(i=1; i \leq N; i--)$
 $\{$
 $\quad \equiv$
 $\}$

$(N > 0)$

$O(\infty)$ Infinite loop

3) $\text{for}(i=1; i \leq N; i *= 2)$
 $\{$
 $\quad \equiv$
 $\}$

$O(\log N)$

1
 $\rightarrow$
 $2 \rightarrow 4 \rightarrow 8 \rightarrow 16 \rightarrow 32 \rightarrow 64 \dots$

4) $\text{for}(i=1; i \leq N; i++)$
 $\{$
 $\quad \text{for}(j=1; j \leq i; j++)$
 $\quad \{$
 $\quad \quad \equiv$
 $\quad \}$
 $\}$

$$\frac{N(N+1)}{2} = \frac{N^2}{2} + \frac{N}{2}$$

$O(N^2)$

i	j	$\# \text{ iterations}$
1	1-1	1
2	1-2	2
3	1-3	3
4	1-4	4
$\vdots$	$\vdots$	$\vdots$
$\vdots$	$\vdots$	$\vdots$
N	1-N	N
		$\frac{N(N+1)}{2}$

3. Importance of constraints

In below if $N \leq 10^{18}$ means in one iteration 10^{18} so we have to write solution in $O(\log n)$

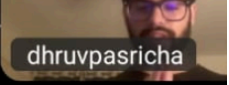
IMPORTANCE OF CONSTRAINTS								$10^7 - 10^8$ iterations in 1 sec	
	$O(N^2)$	$O(N^4)$	$O(N \log N)$	$O(N)$	$O(\sqrt{N})$	$O(\log N)$	$O(1)$		
• $N \leq 10^{18}$	✗	✗	✗	✗	✗	✓	✓		
• $N \leq 10^9$	✗	✗	✗	✗	✓	✓	✓		
• $N \leq 10^7$	✗	✗	✗	✓	✓	✓	✓		
• $N \leq 10^5$	✗	✗	✓	✓	✓	✓	✓		
• $N \leq 10^4$	✗	Maybe	✓	✓	✓	✓	✓		
• $N \leq 10^3$	✗	✓	✓	✓	✓	✓	✓		
• $N \leq 10^2$	✓	✓	✓	✓	✓	✓	✓		

4. Space complexity

1.

SPACE COMPLEXITY

1. Find the **MAX SPACE UTILISED AT ANY POINT** in terms of input size (N)
2. Remove the constants
3. Remove the lower contribution terms



dhruvpasricha

2.

SPACE COMPLEXITY

1. Find the **MAX SPACE UTILISED AT ANY POINT** in terms of input size (N)

2. Remove the constants

3. Remove the lower contribution terms

4 MB

 ✓
 $3 + 2 + 1 + 4 + 2$
 $= 12 \text{ MB}$

12 MB

 ✗

3.

Untitled Notebook 1
Array Techniques (1).pdf

```

int A[N];
for(i=0; i<N; i++)
{
    //
}
        
```

N

TC: $O(N)$

SC:


```

int A[N];
for(i=0; i<N; i++)
{
    //
}

int A

```

$TC: O(N)$
 $SC: O(N)$

Loop $\rightarrow TC$
 Array $\rightarrow SC$

4.

```

int A[N][M];
for(i=0; i<N; i++)
{
    for(j=0; j<M; j++)
    {
        //
    }
}

```

$TC: O(N \times M)$
 $SC: O(N \times M)$

5.

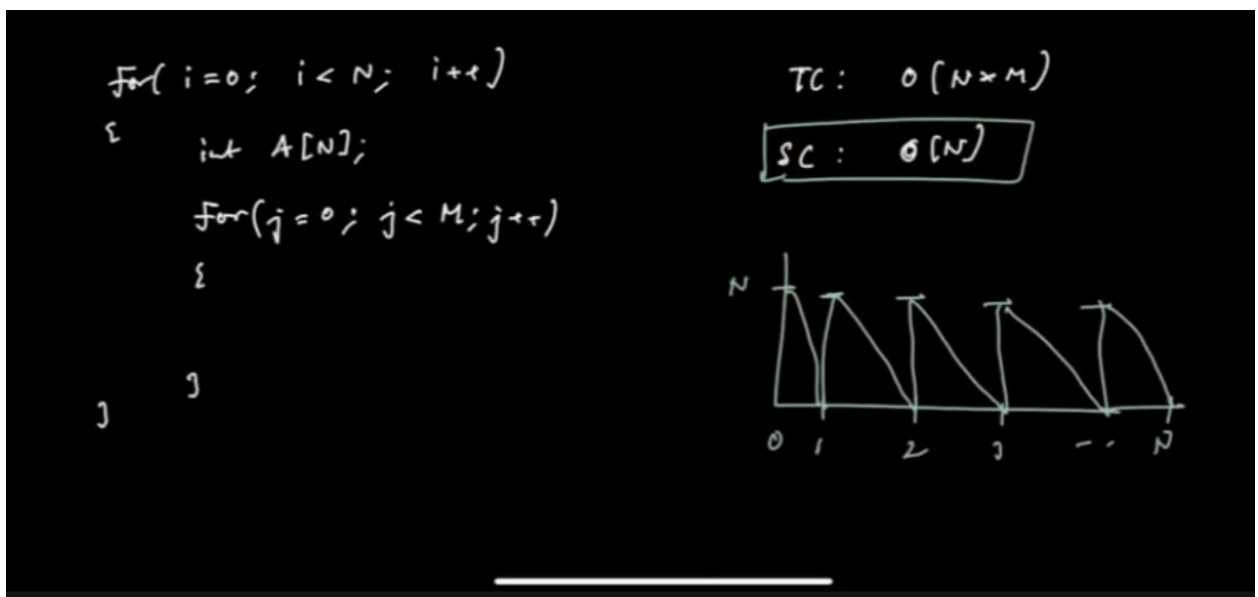
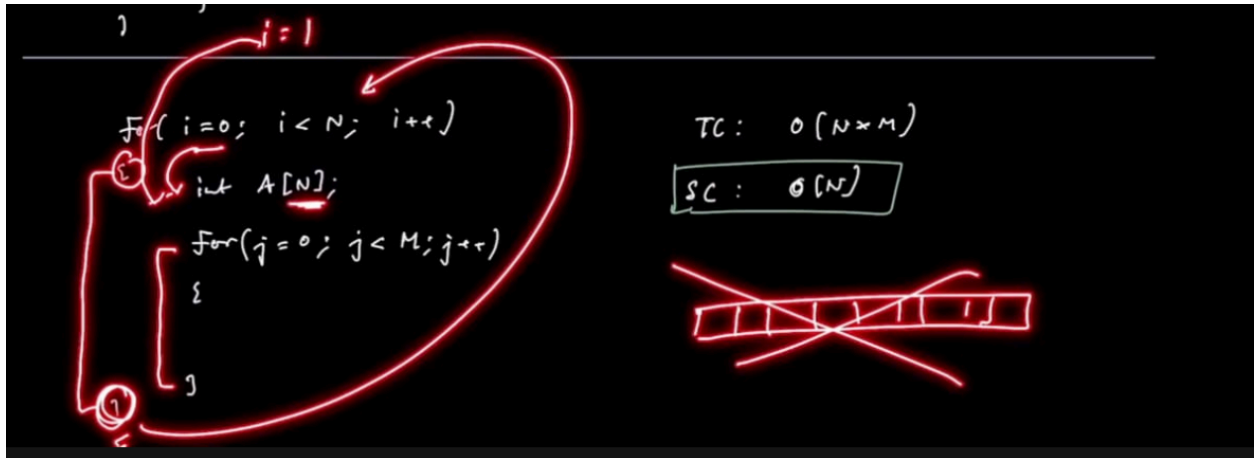
```

for(i=0; i<N; i++)
{
    int A[N];
    for(j=0; j<M; j++)
    {
        //
    }
}

```

$TC: O(N \times M)$
 $SC: O(N)$

Ayush Bhagwat to Everyone
 N



5.

A. Range Sum Query

time limit per test: 1 second
memory limit per test: 256 megabytes

You are given an array $a_1, a_2, \dots, a_n$.

Answer q queries of the form: given l, r , output the **sum** of elements in range $[l, r]$.

Input

The first line contains an integer n ($1 \leq n \leq 10^5$). The second line contains n integers $a_1, a_2, \dots, a_n$ ($|a_i| \leq 10^9$).

The third line contains an integer q ($1 \leq q \leq 10^5$). Each of the next q lines contains two integers l, r ($1 \leq l \leq r \leq n$).

Output

For each query, print the sum of elements in $[l, r]$.

RANGE SUM QUERY

$A = [7, 1, 5, -3, 2, 10, 5, 8]$

0 1 2 3 4 5 6 7

$Ques: L \ R \Rightarrow A[L] + A[L+1] + \dots + A[R]$

1 3 $A[1] + A[2] + A[3] = 3$

0 4 $A[0] + A[1] + A[2] + A[3] + A[4] = 12$



dhruvpasricha

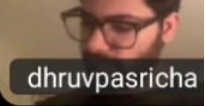
Brute force approach

```

int N;
cin >> N;

int A[N];
for(i = 0; i < N; i++)
{
    cin >> A[i];
}

int Q;
```



dhruvpasricha

8:34 PM Thu 12 Mar

Untitled Notebook 1

Array Techniques (1).pdf

cin >> Q;

for (i = 0; i < Q; i++)

{

i = L, R;

cin >> L >> R;

sum = 0;

for (j = L; j <= R; j++)

{

sum += A[j];

}

print(sum);

}

TC :

$O(Q \cdot N + N)$

$\downarrow$

$O(Q \cdot N)$

$10^5 \times 10^5 = 10^{10}$

TLF

Ekta to Everyone

10^{10}

dhruvpasricha

Optimized approach(Using prefix sum array algorithm)

dhruvpasricha

A = [7, 1, 5, -3, 2, 10, 5, 8]

P = [7, 8, 13, 10, 12, 22, 27, 35]

0 1 2 3 4 5 6 7

$$A = \begin{bmatrix} 7 & 1 & 5 & -3 & 2 & 10 & 5 & 8 \\ 0 & 1 & 2 & 3 & 4 & 5 & 6 & 7 \end{bmatrix}$$

$$P = \begin{bmatrix} 7 & 8 & 13 & 10 & 12 & 22 & 27 & 35 \\ 0 & 1 & 2 & 3 & 4 & 5 & 6 & 7 \end{bmatrix}$$

$$A = \begin{bmatrix} 7 & 1 & 5 & -3 & 2 & 10 & 5 & 8 \\ 0 & 1 & 2 & 3 & 4 & 5 & 6 & 7 \end{bmatrix}$$

$$P = \begin{bmatrix} 7 & 8 & 13 & 10 & 12 & 22 & 27 & 35 \\ 0 & 1 & 2 & 3 & 4 & 5 & 6 & 7 \end{bmatrix}$$

$$P[i] = A[0] + A[1] + A[2] \dots + A[i]$$

L R

$$0 \quad 5 \quad \rightarrow \quad 22$$

$$(L=0) \Rightarrow P[R]$$

$$0 \quad 3 \quad \rightarrow \quad 10$$

$$0 \quad 7 \quad \rightarrow \quad 35$$

$$2 \quad 6$$





dhruvpasricha

$$A = [7, 1, 5, -3, 2, 10, 5, 8]$$

0 1 2 3 4 5 6 7

$$P = [7, 8, 13, 10, 12, 22, 27, 35]$$

0 1 2 3 4 5 6 7

$P[i] = A[0] + A[1] + A[2] + \dots + A[i]$

L R

0	5	→	22	(L=0)	⇒	P[R]
0	3	→	10			
0	7	→	35			
2	6	→	$P[6] - P[1]$			
			$= 27 - 8$			<u>$= 19$</u>

0 L-1 L R



dhruvpasricha

0	3	→	10	0	L-1 L	R
0	7	→	35			
2	6	→	$P[6] - P[1]$			
			$= 27 - 8$			$= 19$

0 L-1 L R

$A[L] + A[L+1] + A[L+2] + \dots + A[R]$

$$\begin{aligned}
 & (A[0] + A[1] + \dots + A[L-1] + A[L] + A[L+1] + A[L+2] + \dots + A[R]) \\
 & \quad - (A[0] + A[1] + A[2] + \dots + A[L-1]) \\
 & = P[R] - P[L-1]
 \end{aligned}$$

Only we have to modify in code below

```

int A[N];
for(i=0; i<N; i++)
{
    cin >> A[i];
}

```

N

```

int P[N];
sum = 0;
for(i=0; i<N; i++)
{
    sum += A[i];
    P[i] = sum;
}

```

N

// Building the Prefix Sum array

```
cin >> Q;
for(i = 0; i < Q; i++) ↵ Q
{
    int L, R;
    cin >> L >> R;

    if(L == 0)
    {
        print(P[R]);
    }
    else
    {
        print(P[R] - P[L-1]);
    }
}
```

Now time complexity

```
8:34 PM Thu 12 Mar
Untitled Notebook 1
Array Techniques (1).pdf
100%

cin >> Q;
for(i = 0; i < Q; i++) ↵ Q
{
    int L, R;
    cin >> L >> R;

    if(L == 0)
    {
        print(P[R]);
    }
    else
    {
        print(P[R] - P[L-1]);
    }
}
```


6.

C. Count Vowels

time limit per test: 1 second

memory limit per test: 256 megabytes

You are given a string s of length n consisting of lowercase English letters.

Answer q queries of the form: given l, r , output the **number of vowels** in substring $s[l \dots r]$. (The vowels are a, e, i, o, u.)

Input

The first line contains an integer n ($1 \leq n \leq 10^5$). The second line contains a string s of length n .

The third line contains an integer q ($1 \leq q \leq 10^5$). Each of the next q lines contains two integers l, r ($1 \leq l \leq r \leq n$).

Output

For each query, print the number of vowels in $s[l \dots r]$.

Examples

input

Copy

$s =$ " a b c d a e f g h i j k l "

0 1 2 3 4 5 6 7 8 9 10 11 12

Queries	L	R	Count	No. of Vowels in L to R
0 4	→	2		'a', 'e', 'i'
3 7	→	2		'o', 'u'
5 12	→	2		

Brute force approach

for every query $[L, R]$: $\leftarrow Q$

count = 0

for $i: L \rightarrow R$ $\leftarrow N$

if $s[i]$ is a vowel;

count++

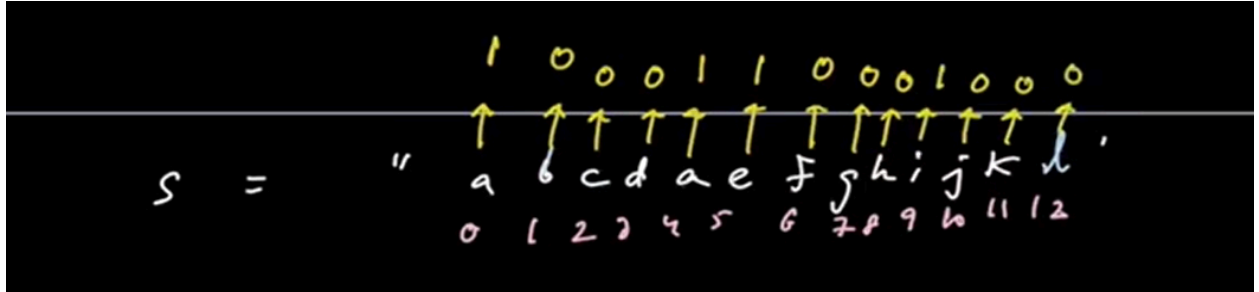
print(count)

TC: $O(Q \times N)$

Optimized solution

Contribution of vowel ex: 'a' -> 1

Contribution of consonant ex: 'b' -> 0



7. One Based index

A. Range Sum Query

time limit per test: 1 second

memory limit per test: 256 megabytes

You are given an array $a_1, a_2, \dots, a_n$.

Answer q queries of the form: given l, r , output the **sum** of elements in range $[l, r]$.

Input

The first line contains an integer n ($1 \leq n \leq 10^5$). The second line contains n integers $a_1, a_2, \dots, a_n$ ($|a_i| \leq 10^9$).

The third line contains an integer q ($1 \leq q \leq 10^5$). Each of the next q lines contains two integers l, r ($1 \leq l \leq r \leq n$).

Output

For each query, print the sum of elements in $[l, r]$.

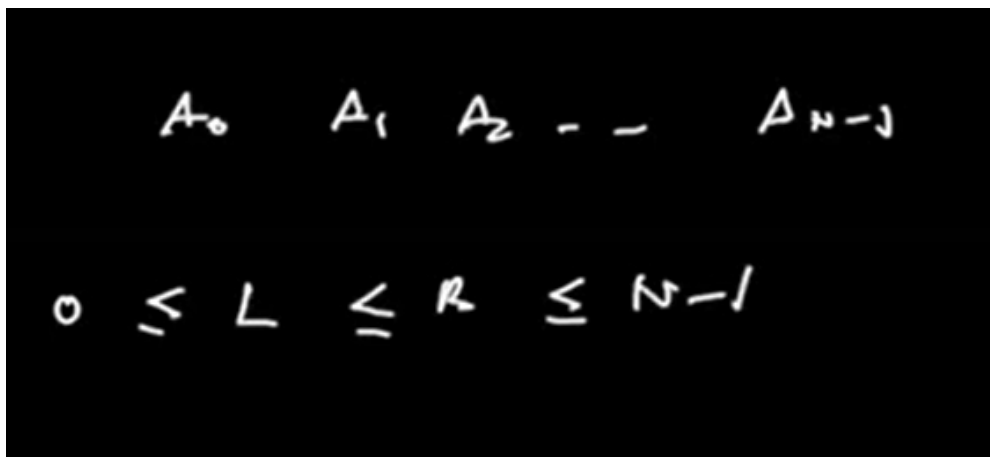
Examples

input

```
5
1 2 3 4 5
3
```

Copy

One based index means -> index start from 1



$A_1, A_2, A_3, \dots, A_N$

$$1 \leq L \leq R \leq N$$

0 based indexing

L starts from 0

$A = [12, 7, 8, 5, 9]$
 0 1 2 3 4

1 based indexing

L starts from 1

$A = [12, 7, 8, 5, 9]$
 1 2 3 4 5

0 based indexing

L starts from 0

$A = [12, 7, 8, 5, 9]$
 0 1 2 3 4

L 2

$(L=0)$

$P[R]$

$(L \neq 0)$

$P[R] - P[L-1]$

1 based indexing

L starts from 1

$A = [12, 7, 8, 5, 9]$
 1 2 3 4 5

```

int L, R;
cin >> L >> R;
if (L == 0)
{
    print(P[R]);
}
else
{
    print(P[R] - P[L-1]);
}

```

SC: $O(N)$

L--;
R--; } } Conw

8.

B. Even Sum Query

time limit per test: 1 second

memory limit per test: 256 megabytes

You are given an array $a_1, a_2, \dots, a_n$.

Answer q queries of the form: given l, r , output the **sum of elements at even indices** in range $[l, r]$. (That is, consider only indices i such that $l \leq i \leq r$ and i is even.)

Input

The first line contains an integer n ($1 \leq n \leq 10^5$). The second line contains n integers $a_1, a_2, \dots, a_n$ ($|a_i| \leq 10^9$).

The third line contains an integer q ($1 \leq q \leq 10^5$). Each of the next q lines contains two integers l, r ($1 \leq l \leq r \leq n$).

Output

For each query, print the sum of elements at even indices in $[l, r]$.

Examples

```

A = [17, 10, 9, 8, 3, 8, 11, 1, 5]
    0  1  2  3  4  5  6  7  8

Queries  L  R  Even Indices

1      6      →  9 + 3 + 11 = 12 + 11 = 23

5      8      →  11 + 5 = 16

```

Solution: Ignore odd indexes by making them 0 then find prefix sum

$A = [12, 10, 9, 8, 3, 8, 11, 7, 5]$
 Indices: 0 1 2 3 4 5 6 7 8

Query 1: L=1, R=6 → $9 + 3 + 11 = 23$ (Note: $12 + 11 = 23$)
 Query 2: L=5, R=8 → $11 + 5 = 16$

Ignore odd indices

9.

First add their square then make prefix

D. Range Sum of Squares
 time limit per test: 1 second
 memory limit per test: 256 megabytes

You are given an array A consisting of N integers. Your task is to answer Q queries of the following form:

For each query containing two integers L and R , compute:

$$A[L]^2 + A[L+1]^2 + \dots + A[R]^2$$

Input

- The first line contains two integers N and Q ($1 \leq N, Q \leq 100000$).
- The second line contains N integers $A_1, A_2, \dots, A_N$ ($1 \leq A_i \leq 10^6$).
- Each of the next Q lines contains two integers L and R ($1 \leq L \leq R \leq N$).

Output

For each query, output a single integer: the value of

$$\sum_{i=L}^R A[i]^2$$

Example

input

```

5 3
2 3 1 4 2
1 3
2 5
    
```

output

```

30
    
```

Handwritten red annotations show the calculation: $1^2 + 2^2 + 3^2 + 4^2 = 1 + 4 + 9 + 16 = 30$.

10. $19 \Rightarrow 1+9 = 10 \rightarrow$ count as 1,
 $10 \Rightarrow 1+0 = 1 \rightarrow$ count as 0

E. Count Numbers with Digit Sum K

time limit per test: 1 second
memory limit per test: 256 megabytes

You are given an array of N integers. You are also given an integer K .

For each query, you are asked to count how many numbers in the array have a digit sum equal to K .

Digit sum of a number x is defined as:

$$\text{digit_sum}(x) = \sum (\text{all digits of } x).$$

Input

- The first line contains three integers N , Q , and K $1 \leq N, Q \leq 100000$, $1 \leq K \leq 100$.
- The second line contains N integers $A_1, A_2, \dots, A_N$ where $1 \leq A_i \leq 10^9$.
- Each of the next Q lines contains two integers L and R $1 \leq L \leq R \leq N$.

Output

For each query, print a single integer representing the number of elements in $[L, R]$ whose digit sum equals K .

Example

input
5 3 10
19 55 10 37 91
1 3
2 5
1 5

output

2
3
4

Handwritten annotations in red:

- Digit sums for the array: 19 (1+9=10), 55 (5+5=10), 10 (1+0=1), 37 (3+7=10), 91 (9+1=10).
- Arrows pointing from the digit sums to the corresponding elements in the array.
- A red circle around the first three elements (19, 55, 10) in the array, indicating they are the only ones with a digit sum of 10.

11. SubArray Concept

Subarrays

↳ Continuous part of an array.

↳ L (starts)

↳ R (ends)

$A = [1, 7, 8, 5, 11]$

0 1 2 3 4

◀ || ▶

L R

0 0

1 1

2 2

3 3

4 4

Subarray

- ↳ Continuous part of an array.
- ↳ L (starts)
- ↳ R (ends)

$A = [1, 7, 8, 5, 11]$
 0 1 2 3 4

L R
 0 0

$L \leq R$

Count number of subArray

$[A_0, A_1, A_2, A_3, A_4, \dots, A_{N-1}]$ Count # of subarray

<u>L</u>	<u>R</u>	<u>#</u>
0	0, 1, 2, ..., N-1	N
1	1, 2, 3, ..., N-1	N-1
2	2, 3, 4, ..., N-1	N-2
⋮		

<u>L</u>	<u>R</u>	<u>#</u>
0	0, 1, 2, ..., N-1	N +
1	1, 2, 3, ..., N-1	N-1 +
2	2, 3, 4, ..., N-1	N-2 +
⋮		⋮
N-1	N-1	1

$$\frac{N(N+1)}{2}$$

Print All subArray

```

A = [ 1, 7, 5, 9 ]

for (L = 0; L < N; L++)
{
    for (R = L; R < N; R++)
    {
        // [L, R]
        for (i = L; i <= R; i++)
        {
            print (A[i]);
        }
        cout << endl;
    }
}
TC: O(N^3)

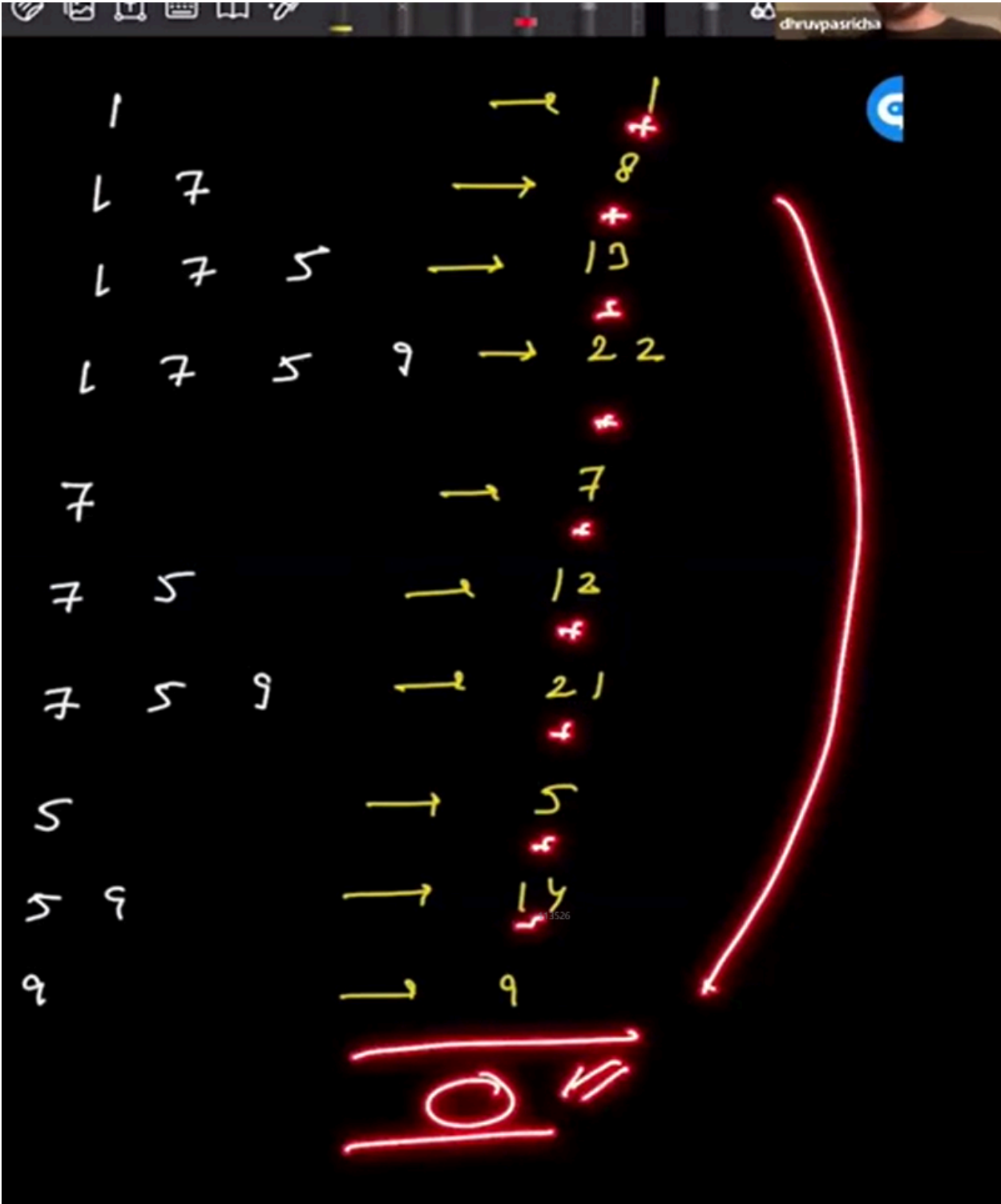
```

```

1
1 7
1 7 5
1 7 5 9
7
7 5
7 5 9
5
5 9
9

```


Sum of all subArray



133% SUM OF ALL SUBARRAYS

ans = 0

for (L = 0; L < N; L++)

{

for (R = L; R < N; R++)

{

sum = 0;

for (i = L; i <= R; i++)

{

sum += A[i];

}

ans += sum

}

}

print(ans)

SUM OF ALL SUBARRAYS

(Google)

ans = 0

for (L = 0; L < N; L++)

{

for (R = L; R < N; R++)

{

sum = 0;

for (i = L; i <= R; i++)

{

sum += A[i];

}

ans += sum

}

}

print(ans)

TC : $O(N^3)$

SC : $O(1)$

Little optimized solution using prefix sum method

```
ans = 0
for (L = 0; L < N; L++)
{
    for (R = L; R < N; R++)
    {
        sum = 0;
        for (i = L; i <= R; i++)
        {
            sum += A[i];
        }
        ans += sum;
    }
}
print(ans)
```

TC: $O(N^3)$
SC: $O(1)$

[L, R]

Sum of $A[L] + A[L+1] + \dots + A[R]$

```
ans = 0
for (L = 0; L < N; L++)
{
    for (R = L; R < N; R++)
    {
        if (L == 0)
        {
            sum = P[R];
        }
        else
        {
            sum = P[R] - P[L-1];
        }
        ans += sum;
    }
}
print(ans)
```

TC: $O(N^2)$
SC: $O(N)$

Optimized solution of sum of all subArray

Number of 1 -> 4

Number of 7 -> 6

Number of 5 -> 6

Number of 9 -> 4

$A = [1, 7, 5, 9]$

(1)
 $(1) (7)$
 $(1) (7) (5)$
 $(1) (7) (5) (9)$
 (7)
 $(7) (5)$
 $(7) (5) (9)$
 (5)
 $(5) (9)$
 (9)

$(1 \times 4) + (7 \times 6) + (5 \times 6)$
 $+ (9 \times 4)$
 $= 4 + 42 + 30 + 36$
 $= 46 + 66$
 $= 112$

$A = [A_0, A_1, A_2, A_3, A_4, \dots, A_{n-1}]$

$ans = (A_0 \times \#subarray \text{ } A_0 \text{ appear})$
 $+$
 $(A_1 \times \#subarray \text{ } A_1 \text{ appear})$
 $+$
 $(A_2 \times \#subarray \text{ } A_2 \text{ appear})$
 $\vdots$
 $\vdots$
 $\vdots$

$$A = \begin{bmatrix} 1 & 7 & 5 & 9 \\ 0 & 1 & 2 & 3 \end{bmatrix}$$

$$\begin{matrix} A_0 & A_1 & A_2 & A_3 \\ = & = & = & = \\ 4 & 6 & 6 & 4 \end{matrix}$$

Lets derive formula

Choose start $\rightarrow$

$$A = [1, 2, 3, 4, 5, 6, 7, 8]$$

Choose end $\leftarrow$

start end

20
4 x 5

$$A = [A_0, A_1, A_2, A_3, \dots, A_i, A_{i+1}, \dots, A_{N-1}]$$

start option = $i+1$

end option = $(N-1) - (i) + 1 = N - i$

subarray = start option x end option

$$A = [A_0, A_1, A_2, A_3, \dots, A_i, A_{i+1}, \dots, A_{N-1}]$$

$$\text{start option} = i+1$$

$$\text{end option} = (N-1) - (i) + 1 = N - i$$

$$\boxed{\text{subarray} = \text{start option} \times \text{end option}}$$

$$\text{ans} = 0;$$

$$\text{for}(i = 0; i < N; i++)$$

$$\{ \text{ans} += (i+1) \times (N-i) \times A[i];$$

$$\}$$

$$\text{print}(\text{ans})$$